What is a Prompt in LangChain?

Prompts are the instructions or queries provided to a model to guide its output. They act as the input layer of communication between the user and the large language model (LLM).



```python
from langchain_anthropic import ChatAnthropic

from dotenv import load_dotenv

load_dotenv()

anthropicModel = ChatAnthropic(model = 'claude-sonnet-4-20250514' , temperature= 0.4 , max_tokens_to_sample =  10)

result = anthropicModel.invoke("Tell me something about India")

print(result.content)
```

Types of Prompts:

Prompts can be broadly categorized into:

1. **Static Prompts**
2. **Dynamic Prompts**

Let's explore both.

Static Prompts

A static prompt is a fixed instruction given to the model. The user needs to rewrite or slightly modify it every time to get a new response.

For example, if you want to ask about India, your prompt could be:

"Tell me something about India."

If you then want to know about Japan or Russia, you would need to rewrite the same prompt and replace the country name.

This approach is:

- **Tedious** (requires repetitive editing).
- **Error-prone** (typos or invalid inputs can cause hallucinations).
- **Not production-ready** (limited flexibility).

That's why static prompts are rarely used in real-world applications.

Example of Static Prompt in Streamlit: Here, the entire input is taken from the user which is classic static prompting.

```
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv
import streamlit as st
load_dotenv()
```

```
st.header("Research Tool")
anthropicModel = ChatAnthropic(
    model='claude-sonnet-4-20250514',
    temperature=0.4
)
input_from_user = st.text_input("Enter your prompt: ")
if st.button("Explore"):
    result = anthropicModel.invoke(input_from_user)
    st.text(result.content)
```

Dynamic Prompts

To overcome the limitations of static prompts, Prompt Templates are used. A prompt template allows you to define a structure with placeholders that get filled dynamically at runtime.
This makes prompts reusable, flexible, and easier to maintain.
Example of a Prompt Template:

“Please summarize the research paper titled "{paper_input}" with the following specifications:
Explanation Style: {style_input}
Explanation Length: {length_input}
1. Mathematical Details:
  * Include relevant mathematical equations if present in the paper.
  * Explain the mathematical concepts using simple, intuitive code snippets where applicable.
2. Analogies:
  * Use relatable analogies to simplify complex ideas.
If certain information is not available in the paper, respond with: "Insufficient information available" instead of guessing.
Ensure the summary is clear, accurate, and aligned with the provided style and length. ”

Instead of rewriting the entire query, the user only provides values for paper_input , style_input, and length_input.

```
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv
import streamlit as st
from langchain_core.prompts import PromptTemplate
load_dotenv()
```

```
st.header("Research Tool")
anthropicModel = ChatAnthropic(
   model = 'claude-sonnet-4-20250514',
   temperature=0.4
)
paper_input = st.selectbox("Select Research Paper Name", [ "Attention Is All You Need", "BERT: Pre-training of Deep Bidirectional Transformers", "GPT-3: Language Models are Few-Shot Learners", "Diffusion Models Beat GANs on Image Synthesis"])

style_input = st.selectbox("Select Explanation Style", ["Beginner-Friendly", "Technical", "Code-Oriented", "Mathematical"])
length_input = st.selectbox("Select Explanation Length", ["Short (1-2 paragraphs)", "Medium (3-5 paragraphs)", "Long (detailed explanation)"])
template = PromptTemplate( template="""
Please summarize the research paper titled "{paper_input}" with the following specifications:
Explanation Style: {style_input}
Explanation Length: {length_input}

1. Mathematical Details:
   * Include relevant mathematical equations if present in the paper.
   * Explain the mathematical concepts using simple, intuitive code snippets where applicable.

2. Analogies:

   * Use relatable analogies to simplify complex ideas.
If certain information is not available in the paper, respond with: "Insufficient information available" instead of guessing.

Ensure the summary is clear, accurate, and aligned with the provided style and length.

""" ,
input_variables=["paper_input" , "style_input" , "length_input"]
)

prompt = template.invoke({"paper_input" : paper_input , "style_input" : style_input , "length_input" : length_input})

if st.button("Explore") :
```

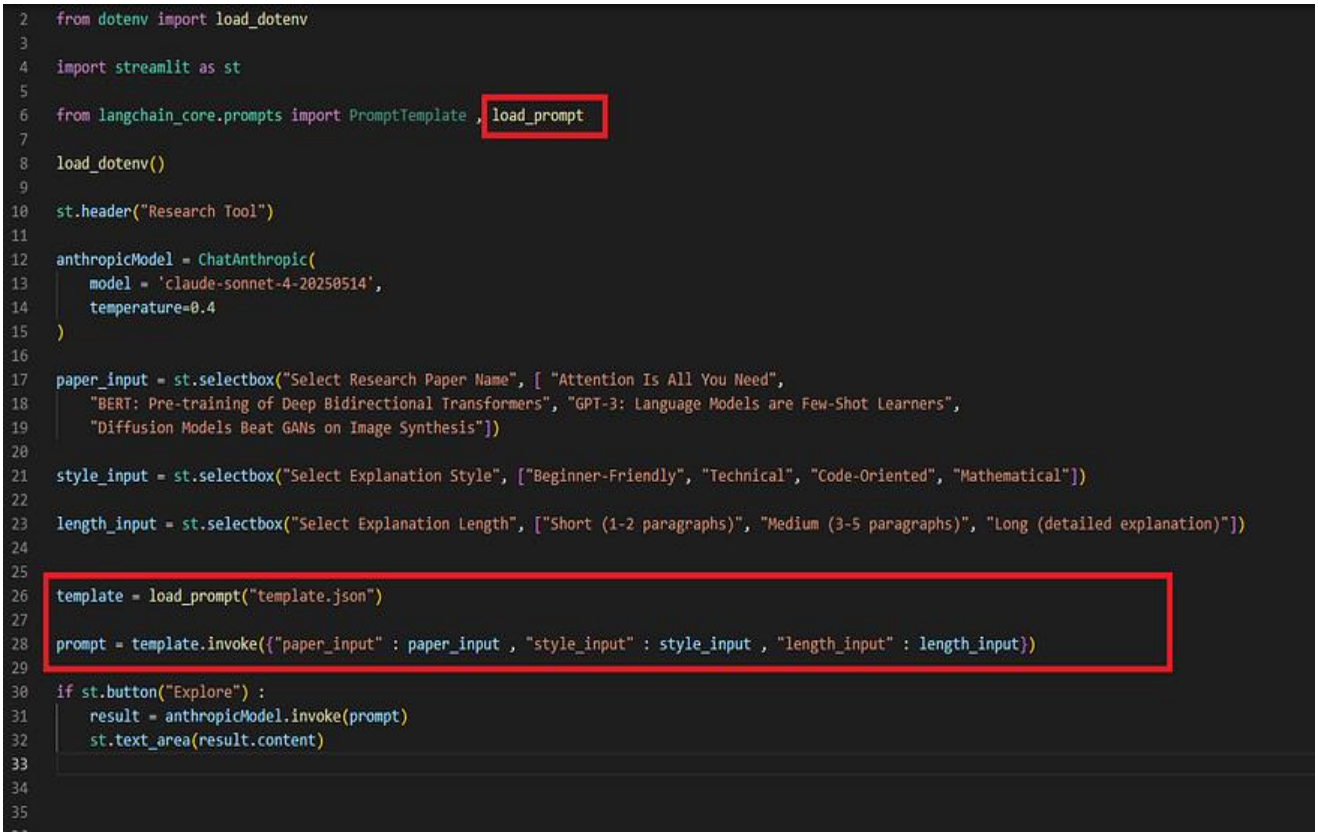
```
result = anthropicModel.invoke(prompt)
st.text_area(result.content)
```

The above code removes repetition, minimizes errors, and improves the user experience. Now by providing these options to users we have avoided the problem of writing our prompt again and again and also there is no margin of having spelling mistakes to avoid the problem of hallucination.

Why Not Use f-Strings Instead?

Instead of promptTemplate we could have also used f Strings (formatted strings in Python).
You could format prompts using Python f-strings, but LangChain's Prompt Template provides several advantages:

1. **Default Validation :** When working with Prompt Template , lets say we provided three inputs to our template and we forgot to provide one of the input then in that case it will raise an error provided we have mention validate_template = True option in the Prompt Template. Also if we provide an extra variable in the input variable section which is not required then also it will raise an error.
2. **Langchain Ecosystem :** It is very compatible with Langchain when working with Chaining.
3. **Reusable :** Lets say we are working in an organization , the template which we have prepared will also be required for other teams for their own purpose then in that case what we can do is , we can create a separate file where we would store this template and save it in json format which will allow others to allow use this template for their own purpose.



```python
from dotenv import load_dotenv

import streamlit as st

from langchain_core.prompts import PromptTemplate , load_prompt

load_dotenv()

st.header("Research Tool")

anthropicModel = ChatAnthropic(
    model = 'claude-sonnet-4-20250514',
    temperature=0.4
)

paper_input = st.selectbox("Select Research Paper Name", [ "Attention Is All You Need",
    "BERT: Pre-training of Deep Bidirectional Transformers", "GPT-3: Language Models are Few-Shot Learners",
    "Diffusion Models Beat GANs on Image Synthesis"])

style_input = st.selectbox("Select Explanation Style", ["Beginner-Friendly", "Technical", "Code-Oriented", "Mathematical"])

length_input = st.selectbox("Select Explanation Length", ["Short (1-2 paragraphs)", "Medium (3-5 paragraphs)", "Long (detailed explanation)"])


template = load_prompt("template.json")

prompt = template.invoke({"paper_input" : paper_input , "style_input" : style_input , "length_input" : length_input})

if st.button("Explore") :
    result = anthropicModel.invoke(prompt)
    st.text_area(result.content)
```

Building a Chatbot with Context:

A simple chatbot without memory looks like this:

```
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv
load_dotenv()
anthropicModel = ChatAnthropic(
   model = 'claude-sonnet-4-20250514',
   temperature=0.4
)
while True :
   user_input = input("Enter your query : ")
   if user_input.lower() == 'exit' :
      break
   result = anthropicModel.invoke(user_input)
   print("Agent : " , result.content)
```

But there are few problems which I would like to show :

```python
from langchain_anthropic import ChatAnthropic

from dotenv import load_dotenv

load_dotenv()

anthropicModel = ChatAnthropic(
    model = 'claude-sonnet-4-20250514',
    temperature=0.4
)

while True :
    user_input = input("Enter your query : ")
    if user_input.lower() == 'exit' :
        break
    result = anthropicModel.invoke(user_input)
    print("Agent : " , result.content)

```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS  NUGET  AZURE  DEBUG CONSOLE

(venv) PS C:\Users\Omkar Patil\OneDrive\Desktop\Langchain-Demo> python '.\4. Prompts\chatbot.py'
Enter your query : hi
Agent :  Hello! How are you doing today? Is there anything I can help you with?
Enter your query : Which one is smaller 2 or 1 ?
Agent :  1 is smaller than 2.
Enter your query : add 10 to the smaller number
Agent :  I don't see any numbers in your message to work with. Could you please provide the two numbers you'd like me to compare? Once you give me the numbers, I'll identify the smaller one and add 10 to it.
Enter your query :
```

In the above screenshot , this lacks context. Each query is treated independently. For example, asking “What’s 5+6?” followed by “and add 5 more” won’t work correctly, since the model doesn’t remember the first message.

To solve the above problem , we can add ChatHistory so that it can lookup on the history chat and answer our questions.

```python
4. Prompts > chatbot.py > ...
anthropicModel = ChatAnthropic(
    model = 'claude-sonnet-4-20250514',
    temperature=0.4
)

chat_history = []

while True :
    user_input = input("Enter your query : ")
    chat_history.append(user_input)
    if user_input.lower() == 'exit' :
        break
    result = anthropicModel.invoke(chat_history)
    print("Agent : " , result.content)
    chat_history.append(result.content)

print(chat_history)
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS  NUGET  AZURE  DEBUG CONSOLE

(venv) PS C:\Users\Omkar Patil\OneDrive\Desktop\Langchain-Demo> python '.\4. Prompts\chatbot.py'
Enter your query : Hi
Agent :  Hello! How are you doing today? Is there anything I can help you with?
Enter your query : which one is smaller 5 or 4?
Agent :  Hello! I'm doing well, thank you for asking!

To answer your question: 4 is smaller than 5.

Is there anything else I can help you with today?
Enter your query : add 5 to the smaller number
Agent :  I'll add 5 to the smaller number, which is 4.

4 + 5 = 9

So the answer is 9.

Is there anything else you'd like me to help you with?
Enter your query : exit
['Hi', 'Hello! How are you doing today? Is there anything I can help you with?', 'which one is smaller 5 or 4?', "Hello! I'm doing well, thank you for asking!\n\nTo answer your question: 4 is smaller than 5.\n\nIs there anything else I can help you with today?", 'add 5 to the smaller number', "I'll add 5 to the smaller number, which is 4.\n\n4 + 5 = 9\n\nSo the answer is 9.\n\nIs there anything else you'd like me to help you with?", 'exit']
(venv) PS C:\Users\Omkar Patil\OneDrive\Desktop\Langchain-Demo>
```

Now we look at the chat_history variable values , there is problem. In future we can have multiple values here and it is not able to identify which is generated by user or

agent information. This would create a problem for LLM to generate output based on this messages to identify whether it was generated by him or by user.

Adding Chat History with Messages:
To fix this, we need chat history. But it's not enough to store messages — we must also track who said what. LangChain provides structured message classes:

- **SystemMessage**: initial instructions.
- **HumanMessage**: user queries.
- **AIMessage**: model responses.

Now lets try to integrate this with our ChatBot:

```
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv
from langchain_core.messages import SystemMessage,HumanMessage,AIMessage
load_dotenv()
anthropicModel = ChatAnthropic(
   model = 'claude-sonnet-4-20250514',
   temperature=0.4
)

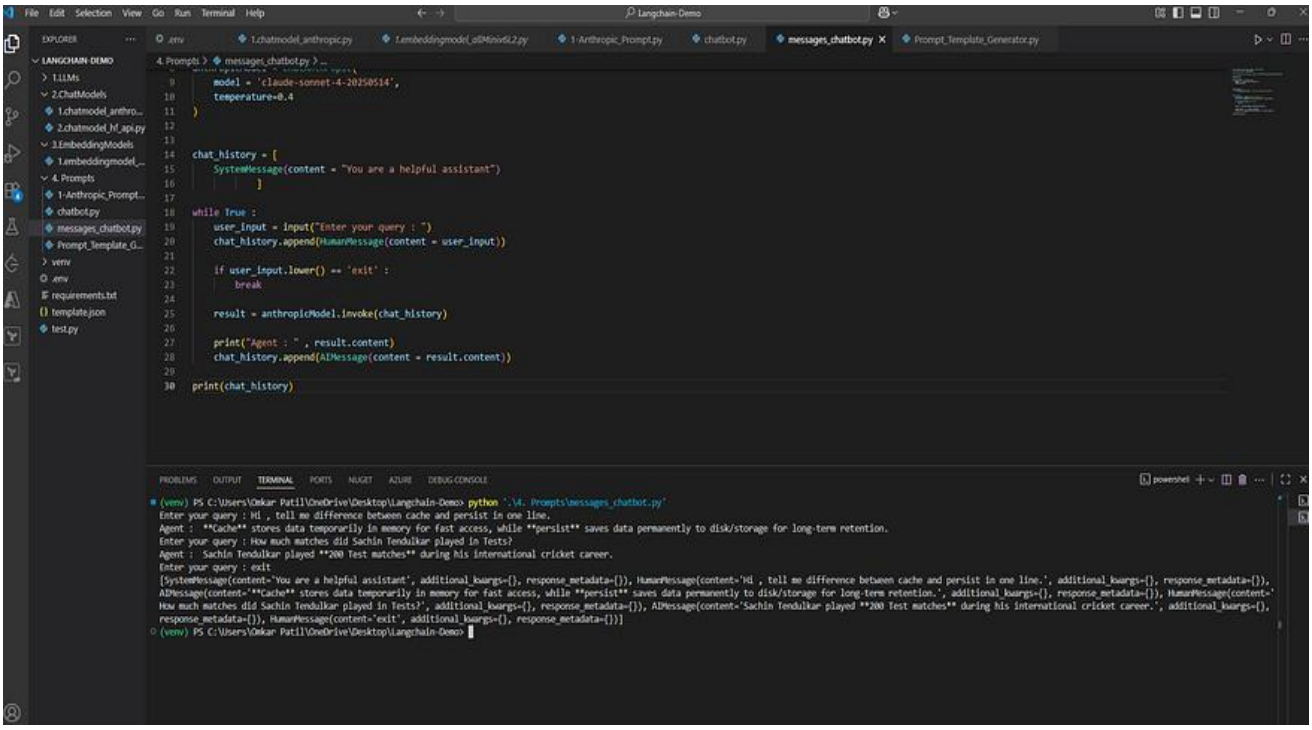

chat_history = [
   SystemMessage(content = "You are a helpful assistant")
        ]

while True :
   user_input = input("Enter your query : ")
   chat_history.append(HumanMessage(content = user_input))

  if user_input.lower() == 'exit' :
      break

  result = anthropicModel.invoke(chat_history)

   print("Agent : " , result.content)
   chat_history.append(AIMessage(content = result.content))
```



Now we have seen how we can send the static query to the list of messages. If we have to work with Dynamic message, then we need to use Chat Prompt Template just like Prompt Template.

Using ChatPromptTemplate and Message Placeholders:

LangChain also supports dynamic chat prompts with placeholders for history.
Here, we use MessagePlaceholder to insert prior messages at runtime.
This allows you to build context-aware conversational agents where the entire chat history is seamlessly passed to the model.

```python
from langchain_core.prompts import ChatPromptTemplate

from langchain_core.messages import SystemMessage,HumanMessage,AIMessage

chat_Template = ChatPromptTemplate([
    SystemMessage(content = 'You are a knowledgeable {domain} expert') ,
    HumanMessage(content= "Explain in simple terms what is {topic}")
])

prompt = chat_Template.invoke({'domain' : 'football' , 'topic' : 'Free-Kick'})

print(prompt)
```

```
(venv) PS C:\Users\Omkar Patil\OneDrive\Desktop\Langchain-Demo> python '.\4. Prompts\messages_chatbot.py'
Enter your query : Hi , tell me difference between cache and persist in one line.
Agent :  **Cache** stores data temporarily in memory for fast access, while **persist** saves data permanently to disk/storage for long-term retention.
Enter your query : How much matches did Sachin Tendulkar played in Tests?
Agent :  Sachin Tendulkar played **200 Test matches** during his international cricket career.
Enter your query : exit
[SystemMessage(content='You are a helpful assistant', additional_kwargs={}, response_metadata={}), HumanMessage(content='Hi , tell me difference between cache and persist in one line.', additional_kwargs={}, response_metadata={}),
AIMessage(content='**Cache** stores data temporarily in memory for fast access, while **persist** saves data permanently to disk/storage for long-term retention.', additional_kwargs={}, response_metadata={}), HumanMessage(content='
How much matches did Sachin Tendulkar played in Tests?', additional_kwargs={}, response_metadata={}), AIMessage(content='Sachin Tendulkar played **200 Test matches** during his international cricket career.', additional_kwargs={},
response_metadata={}), HumanMessage(content='exit', additional_kwargs={}, response_metadata={})]
(venv) PS C:\Users\Omkar Patil\OneDrive\Desktop\Langchain-Demo> python '.\4. Prompts\chat_prompt_template.py'
messages=[SystemMessage(content='You are a knowledgeable {domain} expert', additional_kwargs={}, response_metadata={}), HumanMessage(content='Explain in simple terms what is {topic}', additional_kwargs={}, response_metadata={})]
(venv) PS C:\Users\Omkar Patil\OneDrive\Desktop\Langchain-Demo>
```

Now the problem here is when we passed our input to the ChatPrompt Template , the placeholder were not replaced like how PromptTemplate earlier replaced it. It is mostly confusing and to make this code work we will have to use something different which is present in below code.

```python
from langchain_core.prompts import ChatPromptTemplate

from langchain_core.messages import SystemMessage,HumanMessage,AIMessage

chat_Template = ChatPromptTemplate([
    ('system' , 'You are a knowledgeable {domain} expert') ,
    ('human' , "Explain in simple terms what is {topic}")
])

prompt = chat_Template.invoke({'domain' : 'football' , 'topic' : 'Free-Kick'})

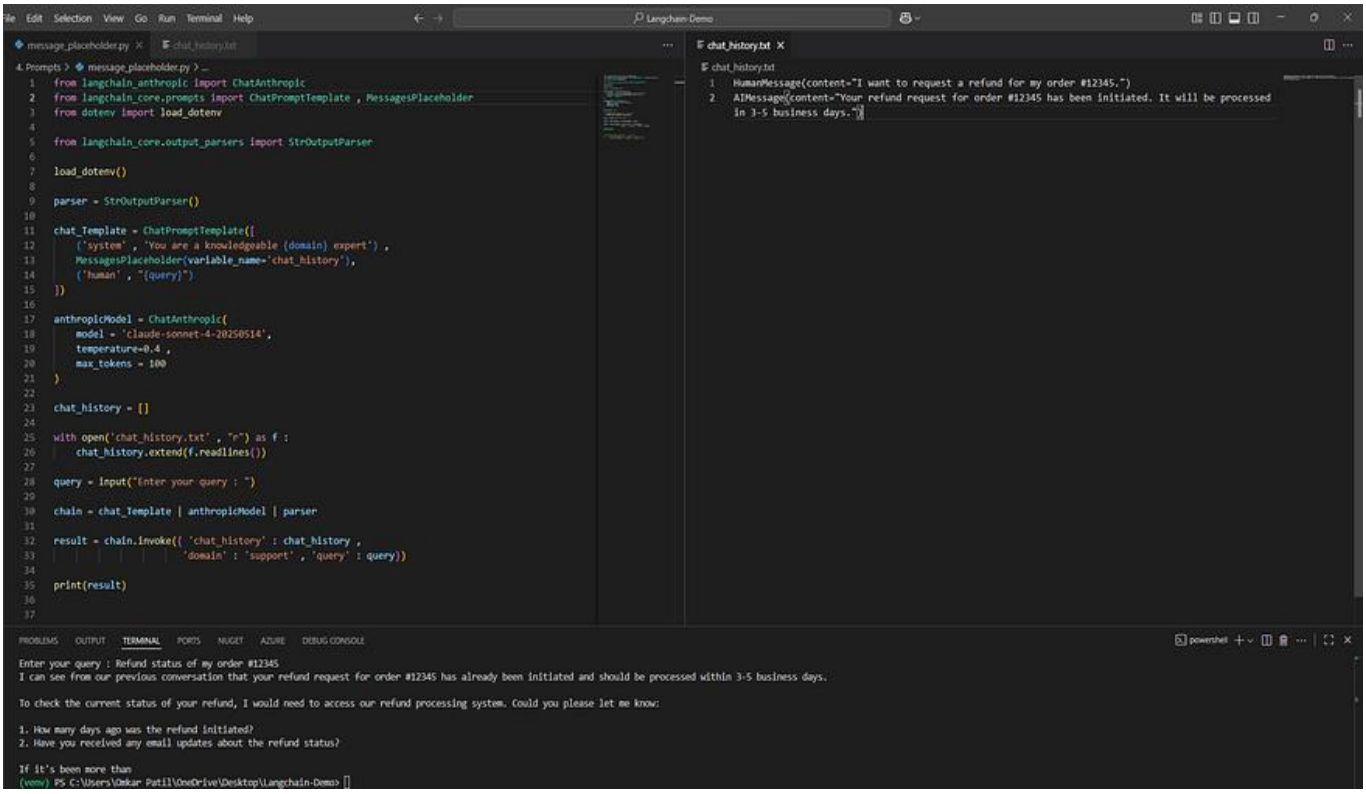
print(prompt)
```

```
(venv) PS C:\Users\Omkar Patil\OneDrive\Desktop\Langchain-Demo> python '.\4. Prompts\chat_prompt_template.py'
messages=[SystemMessage(content='You are a knowledgeable football expert', additional_kwargs={}, response_metadata={}), HumanMessage(content='Explain in simple terms what is Free-Kick', additional_kwargs={}, response_metadata={})]
(venv) PS C:\Users\Omkar Patil\OneDrive\Desktop\Langchain-Demo>
```

We have passed our system and human message in form of tuple which fits the data in our placeholder

What is Message Placeholder?

A message placeholder in LangChain is a special placeholder used inside a ChatPromptTemplate to dynamically insert chat history (a list of messages) at runtime. It allows the model to access the full conversation context without hardcoding past exchanges.



Example of how to implement Message Placeholder in ChatPrompt Template

Example: Using a Message Placeholder in ChatPromptTemplate

Key points from the implementation:

- We import MessagesPlaceholder from langchain_core.prompts
- Inside the ChatPromptTemplate, the chat_history variable acts as the placeholder that holds the entire conversation history
- For demonstration, we created a chat_history.txt file containing a few example messages
- These messages are read and loaded into a chat_history list variable
- Finally, we invoke the chain with our template parameters, and the model automatically incorporates the chat history into its response